RoboCupJunior Rescue (Line) 2026

Team Description Paper

Team Pishnam

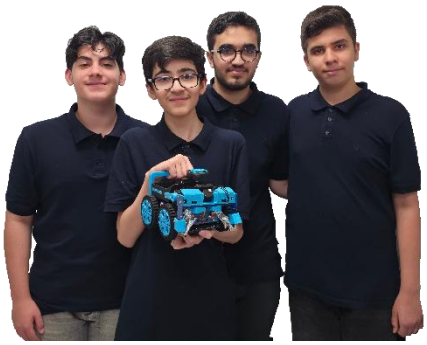

Figure 1. Team Pishnam with the competition robot.

League	RCJ Rescue Line
Age group	14–16
Region	Iran
Institution	Pishnam
Team website	pishnam.com
Mentors	Mohammad Rafiei
Contact	Mohammad Rafiei — pishnam@gmail.com
Date	June 2026

Abstract

Pishnam is an autonomous robot built for the RoboCupJunior Rescue Line challenge and engineered with minimal reliance on ready-made modules. Every electronic board was custom-designed in Altium Designer, and every structural and mechanical part was modelled from scratch in SolidWorks and produced by FDM 3D printing, covering the full path from design to fabrication. The control software is written in C++ for deterministic speed and reliability. The robot follows the line, detects intersections and the evacuation zone reflective silver using an array of 39 infrared sensor pairs in addition to 4 pairs that assist in detecting victims and obstacles, recognizes green intersection markers and the evacuation point colors with three APDS-9960 color sensors, measures wall clearance and obstacles with VL53L1X time-of-flight sensors, and maintains orientation with a CMPS-14 inertial compass. Motion and rescue actions are driven by ten Dynamixel servo motors — six XC430 and four XL330 — coordinated by an ESP32-S3-WROOM-1 microcontroller, with all sensors and actuators integrated across ten team-designed circuit boards. What sets Pishnam apart is the depth of its custom integration: the mechanical, electronic, and software subsystems were each designed in-house and tuned together, yielding a compact, robust platform that performs reliably across the line-following, victim-rescue, and evacuation tasks.

1. Introduction

This paper describes the design and engineering of Pishnam, our entry for the RoboCupJunior Rescue Line league. It covers the team and project plan, the integration strategy, the mechanical and electronic hardware, the control software, and the performance evaluation that guided development.

Designed by Amirfarhan Mohseni

a. Team

Arvin Mirza Aghayan — Software Developer. Three years on the team and four years of robotics experience; represented Pishnam at IranOpen 2025 placing second (Rescue Line Secondary). Proficient in C++, Arvin designs the robot's core algorithms and the control logic for its primary operational systems.

Amirfarhan Mohseni — Mechanical Specialist. Joined this year with four years of robotics and design experience. Skilled in SolidWorks, Amirfarhan is responsible for the full mechanical design. He is also the author of this technical description paper.

Reza Bagheri — Software Developer. Joined this year with five years of robotics experience, including first place at IranOpen 2025 (Rescue Line Primary). Skilled in C++, Reza develops and maintains the control software and supports system integration.

Mohammadamin Arabi — Electronic Specialist. Joined last year with five years of robotics experience, including second place at IranOpen 2025(Rescue Line Secondary). Skilled in Altium designer and Photoshop, he produces the team's circuit boards and competition poster.

Member	Role	Notable result
Arvin Mirza Aghayan	Software Developer	2nd — IranOpen 2025, Rescue Line Secondary
Amirfarhan Mohseni	Mechanical Specialist	1st — RoboCup 2026 Qualifier, Rescue Line Secondary
Reza Bagheri	Software Developer	1st — IranOpen 2025, Rescue Line Primary
Mohammadamin Arabi	Electronic Specialist	2nd — IranOpen 2025, Rescue Line Secondary

Table 1. Team members, roles, and previous competition results.

2. Project Planning

a. Overall Project Plan

Objective. Our objective is to build a fully autonomous robot that follows the Rescue Line course, navigates intersections and obstacles, and rescues victims within the eight-minute run, in full compliance with the official RCJ Rescue Line rules.

Requirements. From the competition rules and our analysis of past runs, we defined the requirements the robot and team had to meet (Table 2):

Requirement	Solution
Reliable line following through gaps, ramps, speed bumps, etc.	wide arch IR array formation with a central gap
Correct detection of green Intersection markers & dead ends	Special APDS-9960 color filter algorithm
Obstacle detection and avoidance	Three VL53L1X`s added to the APDS board
A rescue mechanism that collects victims, distinguishes Injured from dead victims	Designing a three step arm and box rescue mechanism with six motors
A compact, robust chassis the team can fabricate and repair quickly	A single part compact chassis fabricated with PLA+ filament
Easier green detection	Added a pusher blade using a transparent plastic sheet
Detect the reflective silver tape on the evacuation zone entrance	Added four IR pairs to the array making it 39
Adding a handle	3D printed a firm and strong handle
Staying within the eight minute time limit	Sped up the line following, turn and victim rescue process algorithms
Detect evacuation points and they`re colors	Designing and fabricating an APDS board in front of the robot including VL53L1X`s and APDS-9960 color sensors

Table 2. Requirements & Solutions.

Plan and schedule. In 2025, scheduling problems disrupted roughly 80% of our planned tasks. In response we adopted the structured schedule in Table 3: mechanical design (SolidWorks) and electronic design (Altium Designer) were front-loaded, while software development (Arduino/C++) proceeded in parallel on last year's working platform to maximize throughput. Each milestone is assigned an owner and reviewed against its planned date Mechanical and electronic design were scheduled first since software depends on stable hardware; programming cannot be finalized until sensors, actuators, and circuit boards are integrated and tested.

Team workflow. The team meets at the robotics club on Thursdays, Fridays, and additional days as needed. Tasks are tracked in Microsoft To Do and every member updates their progress weekly. Three formal review gates were held: end of October (hardware complete, software begins), end of January (all hardware finalized, software unblocked), and end of April (all modules operational, full-run evaluation before competition prep).

Milestone	Timeline	Owner	Status vs. plan	Conditions Tested
Rescue-mechanism design	Early Aug – Late Aug	Amirfarhan	1 week early	—
Victim-detection process defined	Late Aug – Middle of Sep	Reza & Arvin	Completed	—
Mechanical designs finalized	Early Sep – Middle of Oct	Amirfarhan	2 weeks late	—
Electronic board designs finalized	Early Sep – Early Nov	Mohammad Amin	1 week late	—
3D printing of the robot	Middle of Sep – Early Oct	Amirfarhan	1 week early	—
Full robot assembled	Early Oct – Late Oct	Amirfarhan	1 week early	—
Sensor calibration & testing	Early Nov – Late Nov	Mohammadamin	Completed	—
TDP & poster creation	Early Dec – Early Jan	Amirfarhan & Mohammadamin	Completed	—
Line-following operation	Middle of Jan – Early Feb	Reza	1 week early	Gaps, Speed bumps, Curved lines, Obstacles, Debris
Intersection-detection operation	Early Feb – Middle of Feb	Arvin	Completed	Different Lightings, Debris
Victim-detection operation	Middle of Feb – Middle of Mar	Reza & Arvin	1 week early	All corners, around Obstacle, Evacuation Point Corners
Ramp-navigation operation	Middle of Mar – Late April	Reza & Arvin	1 week late	Speed bumps, curves, Debris
Fake-victim detection operation	Late April – Middle of May	Reza & Arvin	1 week late	Different shapes, Conductivity
Full-run simulations	Middle of May - Ongoing	—	Ongoing	—
TDP revision & finalization	Early June – June 12	Amirfarhan	Completed	
Competition readiness & backup prep	Middle of June – Late June	—	Planned	—

Table 3. Project schedule with milestone owners and progress against the planned timeline.

b. Integration Plan

Component selection is driven by quality, availability, and datasheet support: parts are modelled in SolidWorks and given Altium footprints before fabrication. A 12.6V three-cell (3S) Li-Po battery powers the robot, and voltage regulators derive the 5V and 3.3V rails required by the sensors and the microcontroller. The ten boards are interconnected with IDC ribbon cables, and the ESP32-S3-WROOM-1 on the main board coordinates all sensing and actuation. Figure 2 shows how the core controller links the power system, the sensor suite (infrared, APDS-9960, CMPS-14, VL53L1X), and the actuator and display group (XC430 and XL330 servo motors with a TFT display).

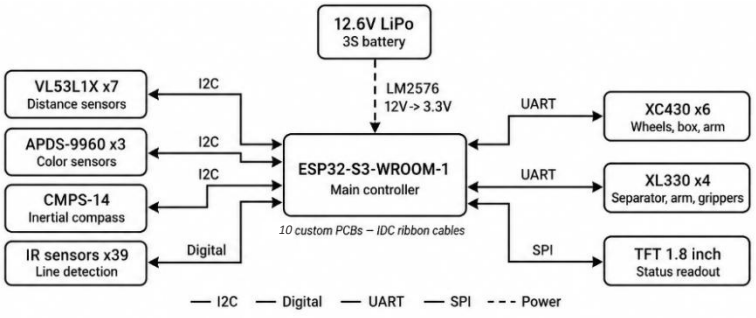

Figure 2. System architecture: the ESP32-S3 core links the

Each subsystem addresses a defined requirement: VL53L1X sensors handle obstacle avoidance, wall-distance control, victim and evacuation-point detection; APDS-9960 sensors handle evacuation-point color and green marker recognition; the CMPS-14 provides compass orientation & ramp detection; the IR array handles line-following and intersection detection; and the Dynamixel motors drive motion, rescue, and victim handling. The redesigned PLA+ rescue box replaced a slow, unreliable earlier mechanism; the evacuation-zone routine was rebuilt through iterative debugging after weak 2025 performance; and the electronics migrated to a single integrated custom design centered on the ESP32-S3, chosen for higher throughput, reliability, and easier iteration.

3. Hardware

Pishnam's hardware comprises two integrated subsystems: a 3D-printed mechanical structure and a fully custom electronic system. The chassis and moving parts are printed in PLA+ and actuated by Dynamixel smart servo motors that connect directly to the robot's circuit boards, which carry the controller, sensors, and power electronics. Figure 3 shows the fully assembled robot.

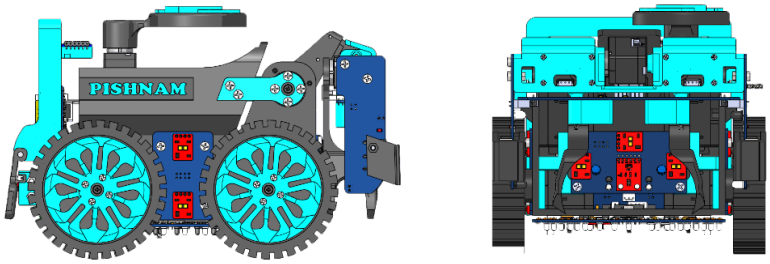

Figure 3. Assembled robot — side view (left) and front view (right).

a. Mechanical Design and Manufacturing

The robot was designed in SolidWorks for precise modelling and fit checking. The entire structure is fabricated from PLA+ filament using FDM 3D printing, covering the chassis, wheels, arm, gripper, and rescue box (Table 4).

Component	Material	Process
Structural frame	PLA +	FDM 3D printing (BambuLab P1S)
Wheels, arm, gripper, box	PLA+	FDM 3D printing (BambuLab P1S)
Tires	EPDM foam	Wrapped over printed wheel

Table 4. Materials and fabrication processes for the main mechanical parts.

- **Structure and fabrication**

The main chassis is built entirely from PLA+ printed parts, chosen for its balance between rigidity and flexibility over PLA, PETG, and TPU, and over plexiglass, wood, and aluminum for faster fabrication, easier iteration, and greater accessibility. Key subassemblies include the wheel mounts, sensor brackets, and arm base — each designed as separate printable modules to allow independent replacement and quick field repairs. One of the most significant mechanical innovations of our design is the integration of the robot's arm within the chassis structure, protecting it from external collisions and improving overall stability during navigation. All parts were 3D printed on a BambuLab P1S 3D Printer using Bambu Studio Slicer.

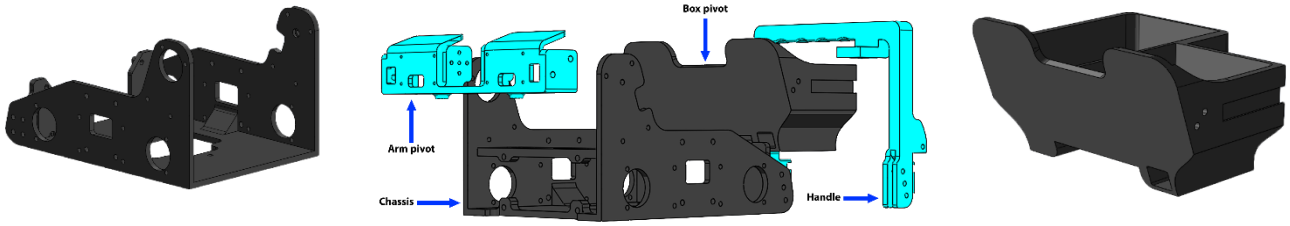

Figure 4. 3D-printed PLA+ chassis (left), Structure Exploded view (middle) & Box (right)

- **Mechanical testing**

Mechanical components were validated through iterative physical testing: the wheels were tested for ramp and speed bump navigation, initially performing inconsistently and requiring tire refabrication with grooves; the rescue box was tested for full rollover across multiple victim drop-offs and redesigned to fully rollover after inconsistent results; and the gripper was tested to maintain grip across different victim sizes without dropping, and was redesigned and reprinted after inconsistent performance. Parts that failed were redesigned and reprinted within one to two days using the BambuLab P1S (Table 5).

Component	Test	Success before & After	Result	Taken Action
Wheels	Ramp/Speed bumps navigation	42/50 → 47/50	Initially Inconsistent	Refabricated tire with grooves
Box	Full rollover test	5/50 → 50/50	Inconsistent	Redesigned to fully rollover
Grippers	Grip Across Different victims	37/50 → 47/50	Inconsistent	Redesigned and reprinted

Table 5. Mechanical component testing results.

- **Rescue mechanism**

The rescue mechanism comprises an arm, a gripper, and a box (Figure 5). The arm collects victims and helps balance the robot on ramps before placing victims in the box. The box is printed as a single PLA+ part, improving print quality and allowing easy replacement if damaged — and is raised by an XC430 servo motor to deposit victims at the evacuation point. The gripper's contact area was enlarged to prevent loss of grip, and a conductivity check at the gripper distinguishes Injured (conductive) from dead victims. Figure 6 shows the gripper securing a victim.

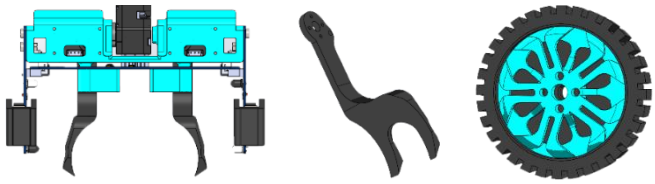

Figure 5. 3D-printed rescue and drive parts: (a) arm, (b) gripper, (c) wheel with EPDM-foam tire.

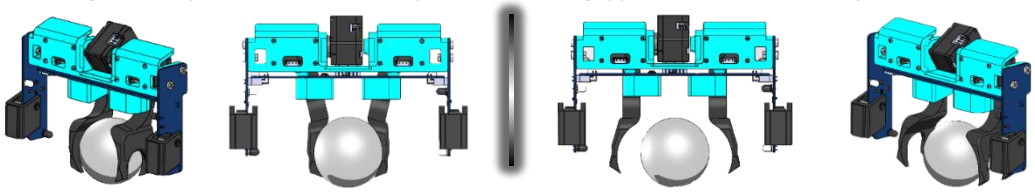

Figure 6. Gripper securing a victim during the rescue sequence.

- **Actuators and power train**

Motion and rescue functions use ten Dynamixel X smart servo motors — six XC430 and four XL330 (Figures 7–8). Both families use the UART-based Dynamixel protocol, so the controller reads feedback and sends commands over just three wires. The 12V XC430 servo motor drive the wheels, arm, and box; the 5V XL330 servos drive the Box separator, Arm Rotation and Grippers (Table 6).

Servo model	Series	Voltage	Used for
XL330-M288-T	Dynamixel X	5V	Box / grippers/ arm rotation
XC430-W240/150-T	Dynamixel X	12V	Arm / box / wheels

Table 6. Dynamixel servo models and their roles.

Figure 8. Dynamixel smart servo motors: XL330 (left) and XC430 (right).

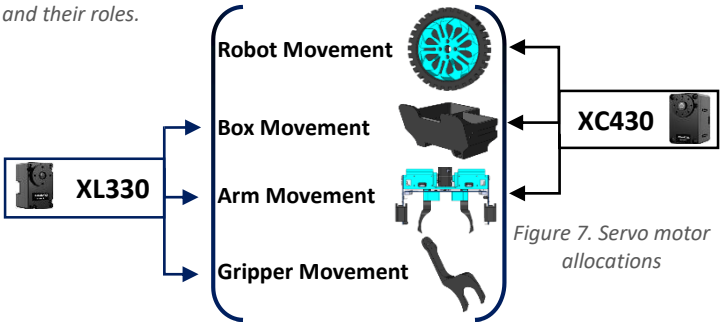

Figure 7. Servo motor allocations

b. Electronic Design and Manufacturing

All sensing and power electronics are integrated across ten boards designed in Altium Designer, selected for its professional toolset. Proven sensors from previous seasons were retained, while outdated parts were upgraded. Every sensor connects to a dedicated board linked to the main board, where the ESP32-S3 manages the system. Table 7 summarizes the sensor suite and Figure 9 shows Mux mapping.

Sensor	Quantity	Interface	Primary purpose
IR sensor pairs	43	GPIO (mux)	Line following, intersections, evacuation-zone entrance, Obstacle victim check
APDS-9960	3	I2C (mux)	Red/green markers, evacuation zone color detection
CMPS-14	1	I2C	Ramp/Seesaw Detection & Precise turn control
VL53L1X (side)	4 (2 per side)	I2C	Wall-clearance in the evacuation zone & Victim Detection
VL53L1X (front)	3	I2C	Obstacle detection and evacuation point detection

Table 7. Sensor suite: quantities, interfaces, and roles.

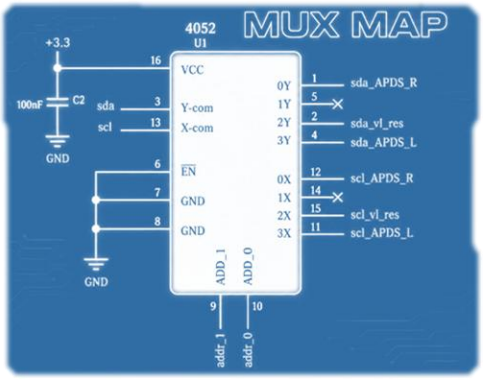

Figure 9. Mux Map

- **Sensors**

Infrared array. An array of 39 infrared sensor pairs forms the robot's primary sensor for line following, intersection detection, and locating the silver evacuation zone marker while 4 additional pairs help with obstacle and victim detection. The pairs are arranged in a wide arch formation with a central gap, covering the front and sides of the robot (Figure 10). Sensor priority is resolved in firmware: the top-center sensors have the highest priority, followed by alternating right-left pairs outward — R1, L1, R2, L2, R3, L3, and so on. The middle and back sensors additionally assist with gap detection and obstacle handling. On curved or angled lines where both outer sensors fire simultaneously, the robot drives straight to avoid losing the line.

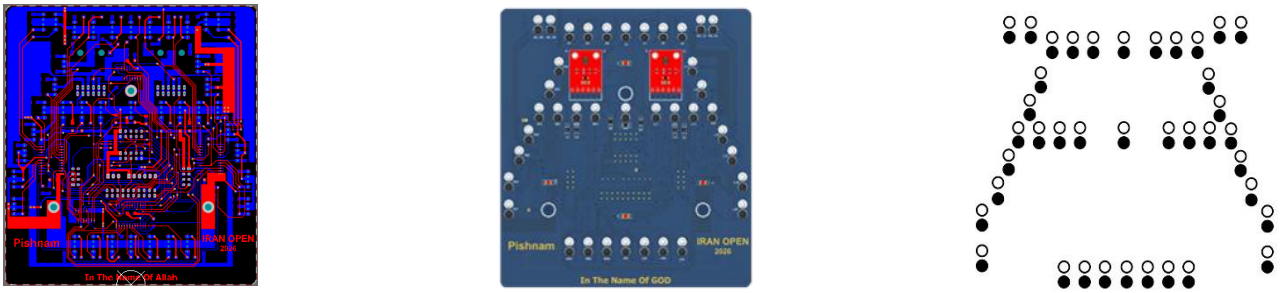

Figure 10. IR sensor array placement.

APDS-9960 color sensors. Three APDS-9960 sensors (Figure 11, left) handle all color-based detection. Two sensors on the sensor board detect green intersection markers and the red finish line; the third, mounted on the APDS board and held by a front-facing chassis mount, identifies the evacuation point color — distinguishing between a green and red evacuation zone to determine the correct victim deposit side.

CMPS-14. The CMPS-14 tilt-compensated compass (Figure 11, right) provides orientation data over I2C (SDA/SCL). It fuses a 3-axis magnetometer, gyroscope, and accelerometer (managed by the BNO080) and corrects tilt-induced error, stabilizing the robot's headings and turns.

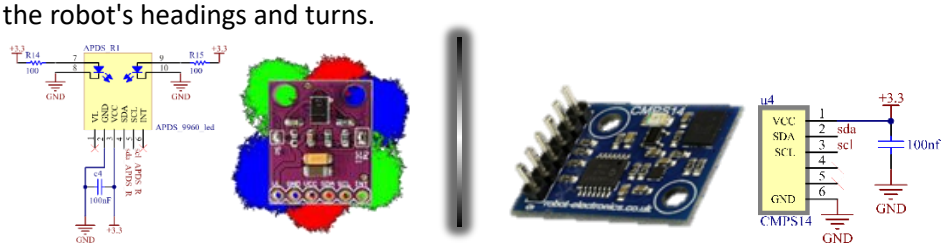

Figure 11. APDS-9960 color detection and schematic (left) and the CMPS-14 compass module and schematic (right).

VL53L1X time-of-flight. VL53L1X time-of-flight sensors handle distance sensing (Figure 12). Two side VL boards maintain wall clearance, side obstacle clearance, and victim detection in the evacuation zone, replacing the earlier SRF05 ultrasonic modules that were physically larger. Three front-facing sensors handle obstacle detection and evacuation point detection, arranged to widen coverage despite the sensor's narrow field of view. Time-of-flight measures the round-trip time of a Class-1 laser pulse, giving fast, accurate, and compact ranging.

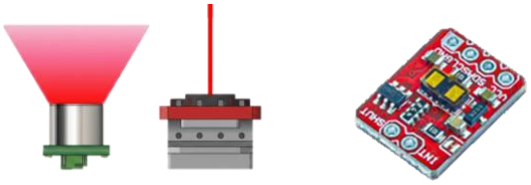

Figure 12. VL53L1X Comparison to SRF05 (left) and the sensor module (right).

- **Actuators**

The Dynamixel XC430 and XL330 servo motors communicate with the ESP32-S3 over UART via the main board, which handles the half-duplex TTL signal conversion required by the Dynamixel protocol (Figure 13).

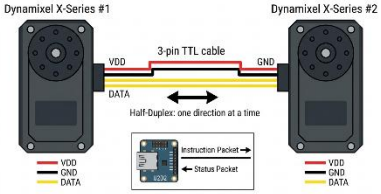

Figure 13. Half duplex TTL signal conversion

- **Battery and Regulators**

A 12.6V three-cell Li-Po powers the robot (Figure 14). Three regulators derive the rails required by the rest of the system (Table 8). Switching regulators are used for their low heat output, with diode, capacitor, and inductor filtering to stabilize the output into a steady voltage.

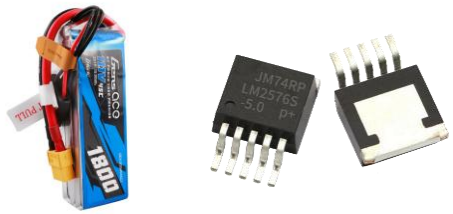

Figure 14. 12.6 V 3S Li-Po battery (left) and the LM2576 step-down regulator (right).

Rail	Regulator	Supplies
12V	Direct from battery	XC430 servo motors
5V	LM2576 (12V → 5V)	XL330 servo motors
3.3V	LM2576 (12V → 3.3V)	ESP32-S3, TFT 1.8``
3.3V	AMS1117 (5V → 3.3V)	Additional 3.3 V components

Table 8. Power rails and the regulators that supply them.

- **Microcontroller**

The ESP32-S3-WROOM-1 — a dual-core processor running up to 240 MHz with integrated Wi-Fi and Bluetooth Low Energy and ample GPIO — was selected over the ATmega2560 and ATmega328p for its far larger memory and processing headroom at low cost (Table 9).

Feature	ESP32-S3-WROOM-1	ATmega2560	ATmega328p
Non-volatile memory	512 KB + ext. EEPROM	4 KB	1 KB
RAM	96 KB	8 KB	2 KB
Programmable pins	36	86	23
Availability	Very high	High	Very high
Price	Low	Medium	Low

Table 9. Microcontroller comparison; the ESP32-S3 was selected for memory and processing headroom.

- **Custom circuit boards**

Ten boards, all designed in Altium Designer and interconnected with IDC cables, integrate the controller, sensors, and actuators (Table 10). Their 3D renders are shown in Figure 15.

Board	Quantity	Key components	Function
Main board	1	ESP32-S3, 1.8” TFT, 4051 mux, 4067 mux, LM2576	Command center; links sensors and motors
Sensor board	1	39 IR pairs, 2x APDS-9960, 4067/4052 mux	Line, and sensing with low pin use
APDS board	1	3x VL53L1X, APDS-9960, 2 IR pairs	Compact evacuation zone, color and obstacle sensing
ESP32 header board	1	ESP32-S3, FM25CL64B F-RAM	Modular controller socket with fast NV memory
Connector board	1	4051 mux, 4-pin connectors	Consolidates wiring and microswitch connectors
Side VL board	2	2x VL53L1X	Repositions/extends the VL53L1X connectors
Right microswitch board	1	Microswitch, 4-pin header, 2 IR pair	Arm contact, Evacuation point detection, alignment
Left microswitch board	1	Microswitch, 4-pin header, 2 IR pair	Arm contact, Evacuation point detection, alignment
Uploader board	1	FT232RL, SS8050G	External programmer kept off the main board

Table 10. The Ten custom Altium boards, their key components, and functions.

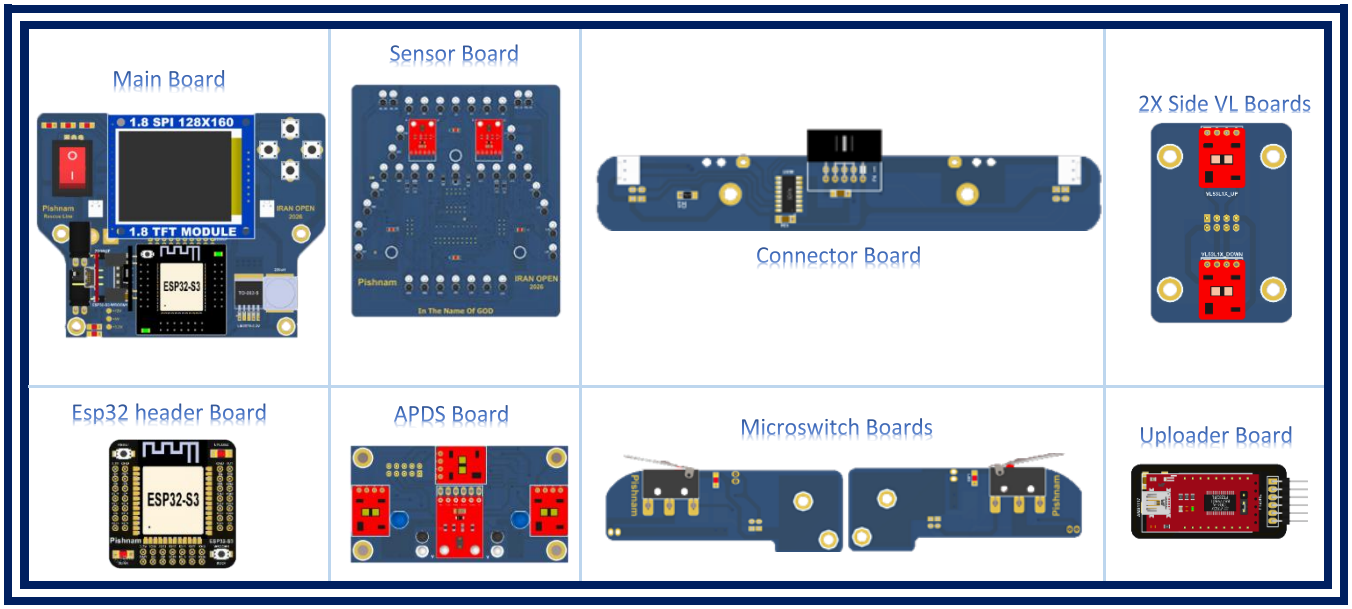

Figure 15. The ten custom boards

- **Electronic testing**

Electronic components were validated through a three-stage testing process: each board was first tested using a multimeter continuity check to verify all connections; track errors found were corrected by hand without requiring board refabrication. Sensors and components were then tested in isolation and all passed without issues. Finally, all ten boards were connected via IDC ribbon cables and tested as a full integrated system; signal interference on the shared I2C bus caused incorrect readings from the VL53L1X and APDS-9960 sensors, resolved by replacing two pull-up resistors and reconfiguring multiplexer pin assignments (Table 11).

Stage	Test	Result	Taken Action
Individual boards	Multimeter continuity test	Track errors found	Manually reworked by hand
Component testing	Sensor reading and communication	All passed	No action needed
Full system integration	All boards connected via IDC cables	I2C bus interference on VL and APDS sensors	placed pull-up resistors and reconfigured multiplexer pin assignments

Table 11. Electronic component testing results.

- **Electronic Innovations**

This year's core electronic innovations were migrating to the ESP32-S3 for its faster processing, larger memory, and built-in Wi-Fi for wireless code uploading, and reducing wiring complexity by plugging the microswitch boards directly into the connector board's female headers — eliminating individual cables and routing a single IDC cable from the connector board

4. Software

The robot’s control software is written in C++ using PlatformIO. PlatformIO was chosen over the Arduino IDE, bare-metal C++, and MicroPython due to its superior performance, faster development workflow, and better project and library management. This overall advantage is shown in Table 12:

Approach	Ease of use	Performance	Control	Exec. speed
PlatformIO	Moderate	High	High	Moderate
Bare-metal C/C++	Moderate	High	High	High
MicroPython	Very high	Low	Low	Low
Arduino IDE (C++)	High	Moderate	Moderate	Moderate

Table 12. Comparison of programming approaches considered for the controller.

a. General Software Architecture

- **Line following, intersections and robot's full control.**

The robot uses a 39-sensor IR array for line following and intersection detection, giving priority to the center sensors and reading the rest in a zig-zag pattern for speed and accuracy. When an intersection is detected, two APDS-9960 color sensors identify the green markers to determine direction with a 96% success rate — their readings cleaned through a custom filter for reliability — while a detected dead end triggers a 180° turn, recognized correctly in 94% of runs. The top IR sensors detect the silver tape and align the robot before it enters the evacuation zone. These behaviors are part of the robot's full control algorithm, which runs as a multi-state machine and is presented as pseudocode in Figure 16, where each sensor condition (line, green marker, ramp, obstacle, victim) maps to an action and transitions the robot between its line-following, obstacle-avoidance, and rescue-zone states.

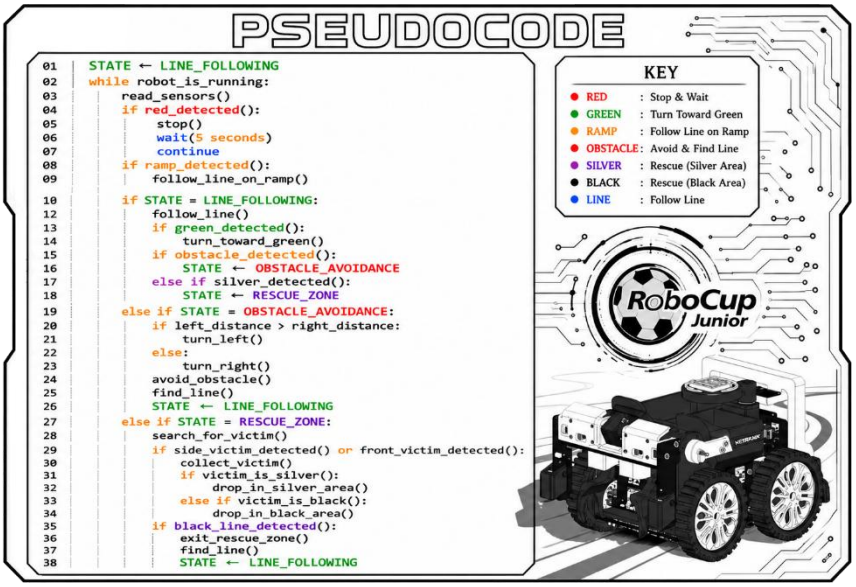

Figure 16. Robot full control PSEUDOCODE.

- **Evacuation zone**

Upon entering the evacuation zone, the robot identifies the furthest wall to orient itself. It then begins a primary search by traversing the perimeter, utilizing two side-mounted VL sensors positioned at different vertical levels (top and bottom). The robot detects victims by continuously comparing the distance readings between these two sensors: if the difference exceeds the known diameter of a victim, the system flags it and navigates toward it for rescue, correctly identifying victims in 94% of runs. This primary perimeter-based search covers approximately 81% of the evacuation zone. If the required number of victims is not localized during this phase, the robot transitions to a secondary search algorithm—illustrated in Figure 17 (Middle)—to systematically explore the interior and clear any remaining areas.

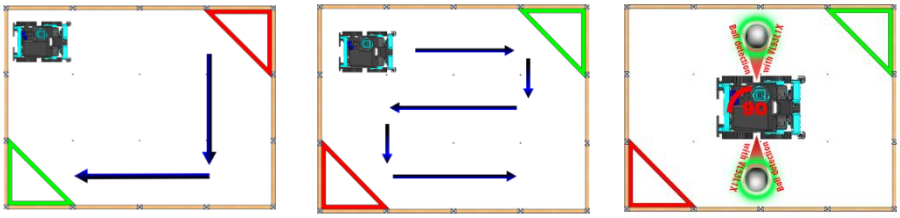

Figure 17. Evacuation-zone scanning and exit-finding strategy.

- **Integration challenge**

During integration, activating the buzzer while switching multiplexer channels caused signal interference; this was resolved by disabling the buzzer before every channel switch, eliminating the conflict entirely.

- **Libraries used**

The software relies on several external libraries alongside the team's custom algorithms. Wire and SPI handle I2C and SPI communication across the sensor suite. The Pololu VL53L1X library manages time-of-flight distance readings, and Adafruit_GFX drives the TFT display. Dynamixel2Arduino handles UART-based servo control using the Dynamixel protocol. WiFi and ArduinoOTA enable wireless code uploading directly to the ESP32-S3 without a physical connection (Table 13).

Library	Wire	Pololu VL53L1X	Dynamixel2Arduino	SPI	Adafruit	WiFi	ArduinoOTA
Usage	I2C Communication	VL53L1X Communication	Motor Communication	SPI Communication	TFT Display	WiFi Connection	WiFi Uploading

Table 13. Software libraries and communication interfaces.

b. Innovative Solutions

- **"Time of no action" Recover**

On squiggling lines, two IR sensors can fire simultaneously and confuse steering. The 'Time of no action' routine was developed after repeated oscillation failures on squiggly segments — briefly ignoring sensor input and driving straight for a bounded interval, so the robot maintains forward progress and recovers the line instead of oscillating (Figure 18). Testing across 50 runs showed a 94% success rate in line recovery, compared to consistent failure before the routine was introduced.

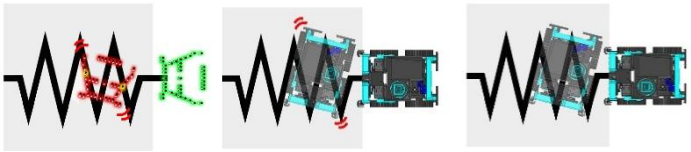

Figure 18. "Time No Act" straight-line recovery on squiggling segments.

- **Dynamic Menu system**

The team developed a dynamic on-board menu system using a TFT display and buttons, allowing real-time configuration and calibration of sensors—such as the IR array—without direct code changes. This achievement significantly improved testing efficiency and the robot's overall adaptability.

5. Performance Evaluation

Pishnam was validated through 50 full runs on practice courses under varied conditions — including different lighting, ramp angles, and obstacle placements. Most issues were identified and resolved before competition, resulting in an overall 62% improvement in run completion rate across the testing period.

Testing procedure. Each run was logged with a pass/fail result per challenge segment. Failed runs were analyzed to identify which module caused the issue, and targeted fixes were applied before the next run cycle.

Results and impact on development. Five recurring failure points were identified and resolved as shown in Table 14:

Challenge	Issue	Fix	Success Rate after fix
Speed bumps on ramps	Getting stuck	Added grooves to the tires	42 out of 50 runs 84%
Obstacle avoidance	Algorithm miss reads	Revised arc algorithm	46 out of 50 runs 92%
Victim detection	Grip Failure	Gripper redesign	47 out of 50 runs 94%
Intersection detection	Detection failure with debris	Added a pusher blade using a transparent plastic sheet	48 out of 50 runs 96%
Evacuation zone	Poor 2025 performance	Full routine rebuild	38 out of 50 runs 76%

Table 14. Performance evaluation results and improvements.

6. Conclusion

Pishnam combines fully custom hardware — PLA+ 3D-printed structures and ten team-designed Altium boards — with a C++ control system to deliver compact, reliable performance across the Rescue Line tasks. Front-loading the mechanical and electronic design while iterating the software in parallel allowed us to correct the timing and evacuation-zone weaknesses of 2025. The team met most of its objectives this season and sees clear opportunities for further refinement in future competitions.

References

- Altium Designer — altium.com
- Robotis Dynamixel e-Manual — emanual.robotis.com
- Bambu Lab — bambulab.com
- SolidWorks — solidworks.com
- SnapEDA — snapeda.com
- GrabCAD — grabcad.com